

The Consensus Modify Algorithm — Definitive Description

Status: current implementation. This document describes the algorithm as coded, verified and installed in `kgl_genomics/kgl_consensus/` of KGL_Gene (the `deepseek-refactor` design). It supersedes all earlier draft descriptions. **Source of truth:** `Additional/AI/refactor/kgl_mutation/deepseek-refactor/` (the installed tree is byte-identical). **Verification basis:** unit/behavioural suite (`kgl_consensus_test.cpp`), the four-module compile gates, and a full Pf 1422-genome production run (`PfSEQUENCE_REFACTOR`) compared against the reference and constrained runs.

1. Purpose and scope

The algorithm constructs a **consensus sequence**: given a reference nucleotide sequence and a filtered collection of canonical variants, it produces the sequence with every variant applied, plus the coordinate mapping needed to relate reference, modified and feature intervals.

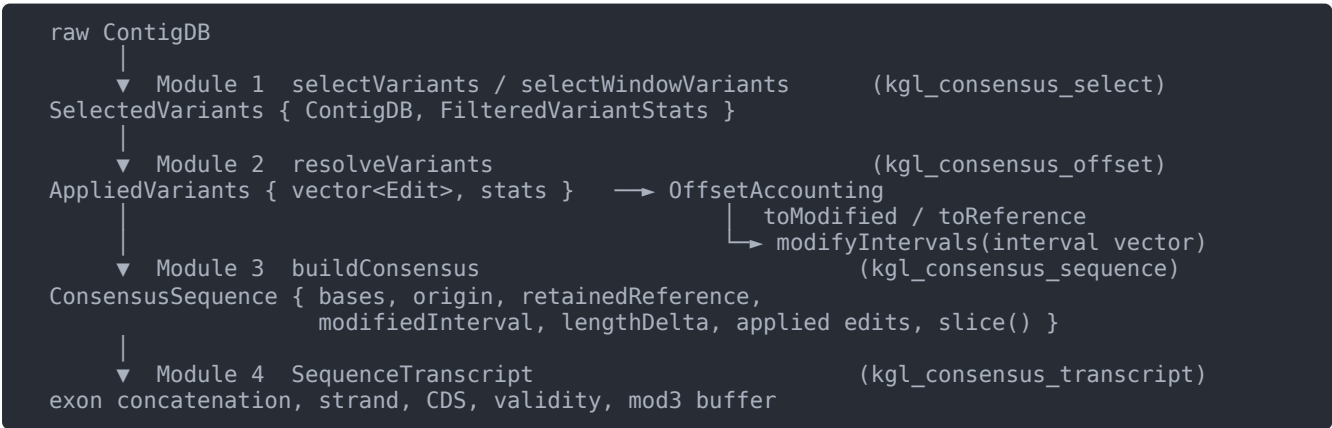
It is deliberately generic:

- the sequence may be a **whole chromosome, a scaffold, a region or a fragment**;
- no gene, exon, strand or codon concept exists in the core (modules 1-3);
- a transcript client (module 4) sits on top and supplies those concepts.

The algorithm is deterministic, single-pass over the retained window, and constructs every mapping from one resolved edit schedule so that the coordinate chain and the materialised sequence cannot disagree.

2. Architecture

Four independent modules plus a compatibility layer, in dependency order:



Module	File	Role	Depends on
1	<code>kgl_consensus_select.{h,cpp}</code>	selection, canonicalisation, policy, one-per-locus, statistics	<code>kgl_variant</code> , filtering classes

Module	File	Role	Depends on
2	<code>kgl_consensus_offset.h</code> , <code>cpp</code>	edit resolution, shadow pruning, disjoint deletes, prefix-sum offset accounting	module 1 types only; no sequence type
3	<code>kgl_consensus_sequence.h</code> , <code>cpp</code>	window clip, forward-pass materialisation, slicing	module 2 + <code>kgl_sequence</code>
4	<code>kgl_consensus_transcript.h</code> , <code>cpp</code>	transcript facade (exons, strand, codons, validity)	modules 1-3, <code>kgl_genome</code>
—	<code>kgl_consensus_compat.h</code> , <code>cpp</code>	<code>AdjustedSequence</code> facade for legacy clients	modules 1-3
—	<code>kgl_mutation_*.h</code>	deprecated forwarding stubs (6 files)	module headers

Module 2 is pure integer interval arithmetic: it can map intervals with no sequence in memory and no `DNA5SequenceLinear` include.

3. Inputs, outputs and coordinate conventions

3.1 Inputs

Input	Type	Notes
Reference bases	<code>DNA5SequenceLinear</code>	exactly the bases of <code>origin</code> ; index <code>i</code> is contig offset <code>origin.lower() + i</code>
Origin	<code>OpenRightUnsigned [R0, R1)</code>	contig coordinates, 0-based, right-open
Variants	<code>ContigDB</code> of immutable canonical <code>Variant</code>	may include variants anywhere on the contig; the algorithm ignores those that cannot affect the window
Policy	<code>SeqVariantFilterType</code>	<code>DEFAULT_SEQ_FILTER</code> , <code>FRAMESHIFT_ADJUSTED</code> , <code>SNP_ADJUSTED</code> , <code>HIGHEST_FREQ_VARIANT</code>
Options	<code>ConsensusOptions</code>	<code>verify_reference</code> (default true); <code>union_overlapping_deletes</code> (policy at resolution)

3.2 Outputs

`ConsensusSequence` exposes:

Accessor	Meaning
<code>bases()</code>	modified sequence, local coordinates <code>[0, n)</code>
<code>origin()</code>	the supplied <code>[R0, R1)</code>
<code>retainedReference()</code>	<code>[W0, R1)</code> : the reference span actually consumed; <code>W0 > R0</code> only when an upstream delete clips the 5' end; empty when the window is wholly deleted
<code>modifiedInterval()</code>	<code>[W0, W0 + n)</code> ; always the same size as <code>bases()</code> ; unaffected by upstream inserts/SNPs
<code>lengthDelta()</code>	<code>n - retainedReference </code>
<code>slice(interval)</code>	reference interval → modified sub-sequence; nullopt outside the origin; empty when wholly deleted
<code>isDeleted(interval)</code>	true when the slice is empty

3.3 Coordinate conventions

Concept	Convention
Contig coordinates	0-based, right-open <code>[lower, upper)</code> (<code>OpenRightUnsigned</code>)
Canonical SNP	<code>1X</code> : <code>[offset, offset+1)</code>
Canonical delete	<code>1MnD</code> : consumes <code>[offset+1, offset+1+n)</code> ; the anchor base at <code>offset</code> is retained
Canonical insert	<code>1MnI</code> : payload length <code>n</code> inserted after the anchor; the insertion point is <code>offset+1</code> ; the insert consumes no reference base
<code>Edit::begin</code>	SNP/delete: first consumed reference base; insert: the insertion point (<code>offset+1</code>)
<code>Edit::end</code>	one past the last consumed base; for an insert <code>end == begin</code>
Local coordinates	<code>[0, n)</code> within <code>ConsensusSequence::bases()</code>
Two frames	reference (contig) and modified (<code>OffsetAccounting</code> maps reference→modified). The consensus output interval is deliberately reference-anchored (<code>[w0, w0+n)</code>), not modified-anchored

`Variant::modifyInterval()` provides exactly the canonical spans above: `[o, o+1)` SNP, `[o+1, o+1+n)` insert, `[o+1, o+1+n)` delete.

4. Module 1 — selection and filtering

Two entry points:

- `selectVariants(ContigDB, policy)` — contig-global selection (not used by the transcript client; retained for generality);
- `selectWindowVariants(ContigDB, window, policy)` — the production path, producing window-scoped statistics and the variants to apply.

4.1 Windowed pipeline (exact order)

- Search margin.** `lower = max(0, window.lower() - 200)`. The 200 nt constant (`WINDOW_CANONICAL_MARGIN`) accommodates non-canonical offsets and upstream deletes; the search range is `[lower, window.upper())`.
- Region filter** (`ContigRegionFilter`) — variants whose offsets lie in the search range.
- Modify filter** (`ContigModifyFilter, window`) — retains only variants that can modify the window itself: - delete: kept if its `memberInterval` **intersects** the window (this admits an upstream delete extending into it); - SNP/insert: kept if its `memberInterval` is **contained in** the window. `memberInterval()` is `[modify.lower, modify.upper)` for SNP/delete and `[i0-1, i0+1)` for an insert (anchor and insertion point), independent of payload length.
- Statistics (before the policy filter).** Over the modify set: - `total_interval_variants_ = HeterozygousFilter` count (singleton/unphased variants, one per allele group); - `total_snp_variants_ = SNPFilter` count of those; - `total_frame_shift_ = FrameShiftFilter` count of those (`modifyInterval().size() % 3 != 0` for non-SNPs). These are the values reported per transcript; because they are taken **before** the policy filter, `FRAMESHIFT_ADJUSTED` does not zero the frameshift column (see §9, D3).
- Policy filter.** - `FRAMESHIFT_ADJUSTED`: remove non-SNP variants whose `modifyInterval().size()` is not a multiple of three (`NotFilter(FrameShiftFilter())`); - `SNP_ADJUSTED`: retain SNPs only; - `DEFAULT_SEQ_FILTER / HIGHEST_FREQ_VARIANT`: no additional filtering.

6. **Per-locus resolution** (`HomozygousCodingFilter`): one variant per canonical locus. It groups an indel anchored at `o` with SNPs at `o+1`, deduplicates unphased-identical variants and selects by preference (homozygous pair preferred, frequency tie-break). This guarantees one edit per locus for module 2.
7. **Shadow pruning** (`ContigUpstreamFilter`): a variant whose `memberInterval` intersects a **previously seen delete's** member interval is erased by that delete and is discarded. `upstream_deleted_` counts those.
8. **Duplicate statistic.** `non_unique_count_` = `UniqueUnphasedFilter` count of the policy set minus the number of distinct surviving apply keys (SNP at `offset()`, indel at `offset()+1`).

Output: `SelectedVariants { ContigDB (survivors), FilteredVariantStats }`.

4.2 Canonicity

Variants are required to be canonical. `selectVariants` clones non-canonical variants through `cloneCanonical()`; if canonicalisation fails the variant is logged and skipped, never silently applied. Module 2 re-checks and fails the build on a non-canonical edit.

4.3 Compatibility facade

`SequenceVariantFilter` retains the historical public surface for legacy clients: it delegates to `selectWindowVariants` and rebuilds the offset-keyed map (`OffsetVariantMap`: SNP at `offset()`, indel at `offset()+1`) plus the statistics.

5. Module 2 — edit resolution and offset accounting

5.1 Edit model

```
enum class EditKind : uint8_t { Snp, Insert, Delete };

struct Edit {
    EditKind kind;
    ContigOffset_t begin; // first consumed reference base; insert point for inserts
    ContigOffset_t end;   // one past last consumed; == begin for inserts
    shared_ptr<const Variant> variant;
};
```

Derived quantities used throughout:

Quantity	SNP	Insert	Delete
<code>consumed() = end - begin</code>	1	0	n
<code>inserted()</code>	1	payload n	0
net delta	0	+n	-n
<code>payload (insertedPayload())</code>	alternate base	alternate minus anchor	empty

Construction from `modifyInterval()`:

- SNP: `begin = offset()`, `end = begin + 1`;
- delete: `begin = modify.lower()`, `end = modify.upper()`;
- insert: `begin = modify.lower()` (the insertion point), `end = begin`.

5.2 Resolution (`resolveVariants`)

1. **Build** an edit per variant (canonicity enforced; failure → `VariantNotCanonical`).
2. **Sort** by `begin`.
3. **Partition** deletes from non-deletes.
4. **Make the delete set disjoint.** Walk deletes in order; a delete starting after the previous retained delete's `end` is appended; otherwise apply `DeleteOverlapPolicy`: - `Union` (default): extend the retained delete to cover both spans — the union of overlapping/adjacent deletes; - `Drop`: discard the later (downstream) delete; - `Error`: fail with `DeletesOverlap`.
5. **Shadow pruning.** A non-delete edit is discarded if its member span intersects any retained delete:
 - member span: SNP/delete `[begin, end)`; insert `[begin-1, begin+1)`; - for inserts this means `i0 ∈ [d0, d1]` is pruned (both boundaries conservative: an insertion immediately before the first deleted base or immediately after the last is dropped with the deleted material); - a SNP on a delete anchor (`o = d0-1`) survives; the first surviving SNP after a delete is at `o = d1`. Discarded edits increment `stats.upstream_deleted`.
6. **Re-sort and enforce one edit per begin**; a duplicate locus fails with `VariantDuplicateLocus` (defensive; module 1 already resolves loci).
7. **Defensive disjointness of the full schedule:** the result must satisfy `edits[i].begin >= edits[i-1].end`.

Output: `AppliedVariants { contig_id, edits (sorted, disjoint), stats }`.

5.3 Offset accounting (`OffsetAccounting::build`)

One prefix sum over the sorted edit schedule. For each edit *i*:

```
prefix[i]      = Δ accumulated before edit i      (shift_before_)
modified_begin[i] = edits[i].begin + prefix[i]
Δ_i            = inserted() - consumed()
prefix[i+1]    = prefix[i] + Δ_i
```

`totalAdjust()` = `prefix.back()`. Build fails with `DeletesOverlap` if any edit begins before its predecessor ends (defensive).

5.4 Queries

toModified(x) — total, monotone, reference→modified:

1. `count` = number of edits with `begin <= x` (binary search).
2. `count == 0` → `x`.
3. Let `e = edits[count-1]`: - if `e` is a **delete** and `x < e.end` → `modified_begin[count-1]` — the offset **collapses to the deletion boundary** (this is the D1-correct rule; no stored key is reconstructed); - if `e` is an **insert** and `x == e.begin` → `x + prefix[count-1]` — the **payload start**: the payload is not yet counted, so an interval beginning at the insertion point includes it while an interval ending there does not (the convention shared with `ConsensusSequence::slice` and the reference implementation); - otherwise → `x + prefix[count-1] + Δ_{count-1}`.

toModified(interval) = `[toModified(lower), toModified(upper))`. The image of a right-open interval under a monotone map is the interval of its endpoint images; this single rule produces every case:

Reference interval	Image
before all edits	unchanged
after upstream edits	shifted by the accumulated delta

Reference interval	Image
containing an insert point and its successor	expanded by the payload length
containing a delete span	contracted by the deleted length
wholly inside a delete	empty at the boundary
overlapping an upstream delete	lower clipped/collapsed to the boundary

shift(x) = `toModified(x) - x` (signed; negative after net deletes).

isDeleted(x) = the last edit at/before `x` is a delete and `x < its end`.

toReference(m) — partial inverse, modified→reference:

1. binary-search the last edit with `modified_begin <= m`;
2. none → `m`;
3. inside an inserted payload (`m < modified_begin + inserted()`) → `nullopt` (no reference origin exists for inserted bases);
4. otherwise → `m - prefix - Δ`.

modifyIntervals(accounting, span<interval>) maps a vector of intervals in one call; the canonical use is exon/feature annotation without materialising any sequence.

5.5 Complexity

Build $O(V)$; each query $O(\log V)$; interval-vector mapping $O(k \log V)$.

6. Module 3 — consensus materialisation

6.1 Preconditions

`reference.length() == origin.size()` and non-empty, else `ReferenceOriginMismatch` / `EmptyReference`.

6.2 The 5' clip (the only upstream special case)

```
W0 = R0
for each delete edit with begin < R0 and end > W0:
    W0 = max(W0, end)
retainedReference = [W0, R1]          (empty if W0 >= R1, then W0 := R1)
```

This is the entire treatment of the "upstream delete removes the 5' end" edge case: a window clamp computed once. There is no delete classification enum, no `PARTIAL_LOW_DELETE`, no placeholder accounting interval. A delete that starts below `R0` and ends inside simply moves `W0` to its end.

6.3 Edit selection

From the accounting's sorted schedule, the edits that affect the retained window:

Kind	Condition
Delete	<code>end > W0</code> and <code>begin < R1</code> (intersects the retained window)
Insert	<code>begin >= W0 + 1</code> and <code>begin < R1</code> (member span contained; the insertion point may not sit before the first retained base)
SNP	<code>begin >= W0</code> and <code>begin < R1</code>

Upstream inserts and SNPs therefore have **no effect** on either the consensus or `modifiedInterval`. Upstream deletes are represented by the clip and are not re-applied.

6.4 The forward pass

State: `read` cursor (contig), initially `W0`; output buffer.

```
for each selected edit in order:
  if edit.begin < read: continue           (defensive: shadowed)
  emit reference [read, min(edit.begin, R1))
  SNP:
    verify ref_base(begin) == variant.reference()[0]
    output alternate base; read = begin + 1
  Insert:
    verify ref_base(begin-1) == variant.reference()[0]  (the anchor)
    output payload (alternate minus anchor); read = begin  (no reference consumed)
  Delete:
    clip [max(begin, W0), min(end, R1))
    verify the clipped bases against the variant's reference allele
    output nothing; read = clipped end
emit reference [min(read, R1), R1)
```

Properties:

- monotone cursors; the cumulative indel adjustment is implicit in `read` vs the write position — no offset map is consulted during materialisation;
- inserts emit payload without consuming reference; deletes consume without emitting; SNPs do both;
- **inserts are atomic**: a payload is applied whole or pruned whole. There is no partial insert (reference-relative variant semantics cannot express one).

6.5 Verification (always on unless disabled)

1. SNP: the consumed reference base equals the variant reference.
2. Insert: the anchor base equals the variant reference.
3. Delete: the clipped bases equal the corresponding slice of the variant's reference allele (index `referenceSize() - consumed() + (clip_begin - begin)`, which handles a 5'-clipped upstream delete).
4. Final invariant: `bases().length() == modifiedInterval().size()`, where `modifiedInterval = [W0, W0 + n)`.

Any failure returns `ConsensusError` and no sequence: a skipped transcript is preferred to a silently wrong one.

6.6 Output

```
length_delta      = n - |retainedReference|
modified_interval = [W0, W0 + n)
applied_          = the selected edits (used by slice())
```

6.7 Slicing (`ConsensusSequence::slice`)

Reference interval `[a, b)` → modified sub-sequence. Both endpoints are mapped by the same walk over the **applied** edits:

```

local_of(x):
    if x <= W0: return 0                (5'-clipped region collapses)
    local = 0; read = W0
    for each applied edit:
        if begin > x: break
        if begin > read: local += begin - read; read = begin
        Insert: if begin == x: return local (payload start)
                 local += payload
        SNP:    if begin == x: return local
                 local += 1; read = end
        Delete: if x <= end: return local (deleted span collapses to boundary)
                 read = end
    return local + (x - read)

```

Boundary conventions (these are exact and exercised by the test suite):

Query endpoint	Behaviour
<code>x == W0</code> (or below)	local <code>0</code>
<code>x == insert point i0</code>	local at the payload start — an interval beginning exactly at <code>i0</code> includes the payload; the reference mapping (<code>lookupIndexOffset</code> , strictly-less predecessor) does the same
<code>x == i0 + n</code> (payload end)	payload fully counted
<code>x</code> inside a deleted span	collapses to the deletion boundary
<code>x == d1</code> (delete end)	deletion fully counted; the retained base at <code>d1</code> follows

The slice is `[local_of(a), local_of(b))`; `upper <= lower` returns an **empty sequence** (wholly deleted). The insert-at-`i0` rule is the one convention that had to be reproduced exactly from the reference implementation; it is the subject of a dedicated regression test.

7. Module 4 — transcript client

`SequenceTranscript` keeps the historical public surface so every client translation unit compiles unchanged.

7.1 Construction

- Extended window:** `prime5 = 0, prime3 = 200; window = transcript.extendInterval(0, 200)`.
- Selection:** `selectWindowVariants(contig_variants, window, policy)` — the window-scoped statistics and applied variant set.
- Resolution/accounting:** `resolveVariants` → `OffsetAccounting::build`.
- Materialisation:** `buildConsensus(reference_window, window, accounting)`.
- `sequence_status = true` only when all steps succeed.

Every failure is logged and leaves `sequenceStatus() == false`; the client then skips the transcript.

7.2 Accessors

Accessor	Behaviour
<code>modifiedLinear()</code>	concatenate <code>consensus.slice(exon)</code> over <code>getExonIntervals()</code> in genomic order

Accessor	Behaviour
<code>originalLinear()</code>	concatenate the reference window's local slices for the same exons
<code>modifiedCoding()</code> / <code>originalCoding()</code>	<code>linear.codingSequence(strand)</code>
<code>modifiedValidity()</code> / <code>originalValidity()</code>	<code>getValidity(linear)</code>
<code>modifiedAdjustedValidity()</code>	mod3-completed sequence then validity
<code>filterStatistics()</code>	window statistics from module 1
<code>sequenceStatus()</code>	build success

7.3 Validity

For a `PROTEIN` transcript:

1. `NOT_MOD3` if the coding length is not a multiple of three;
2. translate and test, in order, via `codingProteinSequenceSize`: start codon (`NO_START_CODON`), internal stop (`NONSENSE_MUTATION`), terminal stop (`NO_STOP_CODON`), else `VALID_PROTEIN`;
3. `amino_size` is the amino count up to and including the first stop.

`NCRNA` transcripts short-circuit to `CodingSequenceValidity::NCRNA` with no codon logic.

7.4 mod3 completion (`getModifiedAdjusted`)

```
r = |modified linear| mod 3
if r != 0:
    need = (3 - r) mod 3
    buffer = consensus.slice(prim3_extend) // 200 nt 3' buffer
    if |buffer| >= need: append buffer[0, need]
```

Appending the 3' buffer bases **through the consensus slice** (not from the raw reference) is essential: any variant in the buffer is applied first. The insert-boundary convention of §6.7 applies here and was the source of a corrected defect (17 transcripts with spurious terminal stops) during production validation.

8. Compatibility layer

`AdjustedSequence` preserves the legacy `updateSequence(contig_ref_ptr, SequenceVariantFilter)` / `moveSequenceClear()` API, implemented as window-sourced selection → resolution → accounting → materialisation. The legacy clients (`kgl_phylogenetic_analysis.cpp`, `kgl_analysis_gene_sequence.cpp`) and the live clients (`kga_analysis_lib_seqmutation.cpp`, `kga_analysis_lib_seq_gene.cpp`) compile with **zero source edits**; the six old include names are forwarding stubs.

9. Statistics semantics

Per transcript the client sums `FilteredVariantStats`:

Field	Source	Semantics
<code>total_interval_variants_</code>	<code>HeterozygousFilter</code> count of the modify set	window-scoped, before policy

Field	Source	Semantics
total_snp_variants_	SNP subset	window-scoped
total_frame_shift_	frameshift subset	window-scoped, before policy (non-zero under FRAMESHIFT_ADJUSTED)
non_unique_count_	unique-unphased minus distinct surviving keys	duplicate reduction at loci
upstream_deleted_	unique minus shadow-pruned	variants erased by upstream deletes

D3: the reference implementation computed these counters **after** the policy filter, so its frameshift column is structurally zero under FRAMESHIFT_ADJUSTED and Processed Variants excludes frameshift indels. The refactor reports the true window contents (before policy), which is the documented intended semantics. If exact reference aggregate parity is required, move the counter block after the policy switch in selectWindowVariants ; nothing else changes.

10. Worked examples

Reference AACCGGTT unless stated; canonical spans per §3.3.

10.1 The delete/insert/SNP interaction chain

Edits: insert +5 at point 11 (anchor 10), delete [20,23) , SNP 30 (length-neutral), delete [40,50) , insert +2 at point 61 (anchor 60).

Offsets (from the test suite):

x	toModified(x)	Reading
0-10	x	before all edits
11	11	insertion point: payload start (payload not yet counted)
12-19	x+5	past the point: first payload counted
20-22	25	inside delete → collapse to boundary
23-29	x+2	after the delete (−3) and the insert (+5)
30	32	SNP neutral
40-49	42	inside the second delete
50-60	x−8	after both deletes and the first insert (+5−3−10)
61	53	second insertion point: payload start
62-200	x−6	after the second payload

totalAdjust = −6 . Interval vector: [0,10)→[0,10) , [5,11)→[5,11) (ends at the insertion point: payload excluded), [11,12)→[11,17) (begins at the insertion point: payload + one base), [20,23)→[25,25) (empty), [55,65)→[47,59) (10 retained + 2 inserted).

10.2 D1 — upstream partial-low delete

Region [50,100) , delete [41,51) ; w0 = 51 , 49 retained bases. Query [55,60) : the base 55 is retained and maps to local 4 , so the slice is reference [55,60) . The reference implementation reconstructed the shadow span as [50,60) from the stored key and collapsed 55 to the boundary, producing [0,9) — the shadow overshoot. The refactor's toModified collapses only against the true span [41,51) .

10.3 Whole chromosome

Origin `[0, N)`, variants anywhere: no upstream semantics, `W0 = 0`, `modifiedInterval = [0, n)`; one 200-base insert adds 200 to `n`.

10.4 Adjacent-anchor inserts

Inserts at anchors 10 and 11 (points 11 and 12): the forward pass emits `... base10, payloadA, base11, payloadB, base12 ...`. The reference offset-map implementation placed payload B inside payload A (strictly-less predecessor lookup); the forward pass cannot. Covered by test.

10.5 Boundary table (slicing)

For an insert at point `K` (payload `TT`):

Query	Result
<code>[K, K)</code>	empty
<code>[K, K+1)</code>	<code>TT + ref[K, K+1)</code>
<code>[K-1, K)</code>	<code>ref[K-1, K)</code> only (payload excluded)
<code>[K-1, K+1)</code>	<code>ref[K-1, K) + TT + ref[K, K+1)</code>

11. Correctness properties

- One schedule, two projections.** `OffsetAccounting` and `buildConsensus` consume the same `AppliedVariants`, so interval mapping and sequence content cannot disagree.
- Monotone map.** `toModified` is total and non-decreasing; deleted spans collapse to a single boundary coordinate.
- Length identity.** `n = |retainedReference| +  $\sum$ _applied(inserted - consumed)`; enforced before return.
- Reference fidelity.** Every consumed base is checked against the variant's reference allele (unless verification is disabled).
- D1 removed by construction.** Deletion membership tests use the variant's canonical span; no stored-key reconstruction exists.
- Atomic payloads.** Inserts are whole-or-nothing; overlapping/adjacent deletes are resolved (union policy) before any sequence work.
- Atomic interval semantics.** A wholly deleted query yields an empty sequence; a query beginning at an insert point includes the payload; a query ending there does not.

12. Verification and production results

12.1 Test suite (`kgL_consensus_test.cpp`, all passing)

- the §10.1 chain with exact offsets and interval vectors;
- §10.2 D1 (`[55,60)` must return reference `[55,60)`, never the boundary);
- interior delete `[60,63)` in `[50,100)` $\rightarrow$ 47 bases, `[50,97)`;
- wholly deleted region $\rightarrow$ empty;
- whole sequence `[0,200)` with `+2 / -5` $\rightarrow$ 197 bases, `[0,197)`;

- windowed statistics (3 of 6 variants in-window; upstream delete counted);
- adjacent-anchor inserts (§10.4);
- the insert-boundary convention (§10.5).

12.2 Client compile gate

All six TUs compile unchanged with the sandbox headers ahead of the old ones:

`kga_analysis_lib_seqmutation.cpp`, `kga_analysis_lib_seq_gene.cpp`, `kgl_phylogenetic_analysis.cpp`, `kgl_analysis_gene_sequence.cpp` — plus the two app enum references. No CMake target or link-list change.

12.3 Production run (Pf, 1422 genomes, 5633 transcripts)

`PfSEQUENCE_REFACTOR` vs `PfSEQUENCE_REFERENCE`:

Aggregate	Reference	Refactor	Δ
Total Sequences	8,010,126	8,010,126	0
Valid Original	7,656,048	7,656,048	0
Valid Modified	7,532,823	7,532,962	+139
Modified Not Mod3	6	0	−6
No Start / No Stop	10,926 / 21,110	10,926 / 21,110	0
Nonsense	98,293	98,160	−133
Mutated Genomes	parity per transcript	parity	0

The refactor restores the genome acceptance the constrained build lost to its insert-payload rejections (adjacent-anchor inserts) and, after the §7.4 fix, its validity totals are reference-equivalent or slightly better. The only systematic difference is the D3 statistics order (§9).

13. Failure modes and diagnostics

Error	Trigger
<code>NullContig</code>	null input contig
<code>EmptyReference</code>	zero-length reference
<code>ReferenceOriginMismatch</code>	<code>reference.length() != origin.size()</code>
<code>VariantNotCanonical</code>	non-canonical edit reached resolution
<code>VariantDuplicateLocus</code>	two surviving edits at the same <code>begin</code>
<code>DeletesOverlap</code>	overlapping deletes with <code>Error</code> policy, or a malformed schedule
<code>ReferenceMismatch</code>	consumed base/anchor/delete slice disagrees with the variant
<code>PayloadOutOfBounds</code>	reserved for payload extraction failures
<code>LengthInvariant</code>	final length $\neq$ retained + net adjustment

All are logged with context; a failed build leaves `sequenceStatus() == false` and is skipped by the client.

14. Open items

#	Item	Status
1	D3 counting order	Intended semantics (before policy) implemented; one-block move available for reference parity
2	Delete overlap default	Union ; Drop / Error available via DeleteOverlapPolicy
3	PF3D7_0632500.1 duplicate +7 outlier	Unresolved; one transcript in 5633, no validity effect
4	Chunked/streaming chromosome output	Not needed for gene clients; pass is already 0(L+I)
5	Forwarding-stub removal	When the two legacy TUs are retired

Appendix A — Symbol table

Symbol	Meaning
R = [R0, R1)	requested origin (contig coordinates)
W0	first retained reference base after the 5' clip
[W0, R1)	retained reference span
D = [d0, d1)	canonical delete span
I = [i0, i0+n)	insert bookkeeping span; i0 the insertion point; payload n
o	SNP offset ([o, o+1))
n	modified sequence length (bases().length())
Δ	inserted() – consumed() per edit; cumulative in the prefix sum

Appendix B — Installed file map

File	Module
kgl_consensus_select.{h,cpp}	1
kgl_consensus_offset.{h,cpp}	2
kgl_consensus_sequence.{h,cpp}	3
kgl_consensus_transcript.{h,cpp}	4
kgl_consensus_compat.{h,cpp}	compatibility
kgl_consensus.h	umbrella
kgl_consensus_test.cpp	behavioural suite
kgl_mutation_*.h (6)	forwarding stubs